

Quicksort (odkrywca: Tony Hoare, 1959)

Technika "dziel i zwyciężaj"

Sortujemy podtablicę $A[L..R]$ tablicy $A[0..n-1]$.
Metoda Hoare'a:

- cel: poprzesuwać element podtablicy tak, by dla pewnego p , $L \leq p < R$ zachodziło $A[i] \leq A[j]$, dla dowolnych $i=L, \dots, p$, $j=p+1, \dots, R$;
- ustal element dzielący $v \in A[L..R]$ ("pivot");
- idąc wskaźnikiem i od lewej szukaj $A[i] > v$;
- idąc wskaźnikiem j od prawej szukaj $A[j] < v$;
- jeśli $i < j$ to (inwersja) zamień $A[i]$ z $A[j]$ i kontynuuj przesuwanie wskaźników;
- jeśli $i \geq j$, to ustal p - miejsce podziału, w pobliżu i , j , tak by $L \leq p < R$ oraz zachodziło:
$$A[i] \leq v, i=L, \dots, p, \quad A[i] \geq v, i=p+1, \dots, R.$$
- posortuj oddzielnie podtablice $A[L..p]$, $A[p+1..R]$ (rekurencja). KONIEC.

Hoare nie podał szczegółowego kodu algorytmu, w szczególności jak dokładnie ustalać miejsce p .

Istnieje wiele różnych realizacji technicznych algorytmu podziału tablicy. Tutaj podajemy wersję Sedgewicka.

Znajomość tej wersji **OBYWIAZUJE**.

```
// Quicksort wg Sedgewicka
main
    A[n] = max (A[0..n-1]);
    // kopia największego w tablicy, wartownik
    quicksort(A, 0, n-1);
end.
```

```

quicksort(A, L, R)
    if (L < R)
        p = Partition(A, L, R); // A[p] - pivot
        Quicksort(A, L, p-1);
        Quicksort(A, p+1, R);
end;

partition (A, L, R)
// elementem dzielącym jest A[L] - pierwszy od lewej
// założenie: wartownik A[R+1] >= A[i], i=L,...,R
i = L; j = R+1; v = a[L];
while (TRUE)
    while (A[++i] < v); // szukaj A[i]>=v
    while (A[--j] > v); // szukaj A[j]<=v
    if (i >= j) break; // i,j minęły się
    swap (A[i], A[j]);
    // zamień, szukaj następnej pary
swap (A[L], A[j]);
// pivot wpada na swoje ostateczne miejsce
// i nie wchodzi w zakresy podzadań
return j; // miejsce podziału
end.

```

Obserwacja:

Elementy równe pivotowi również są zamieniane.

Fakt.

Algorytm quicksort nie jest stabilnym algorytmem sortowania.

Przykład:

indeksy elementów tablicy A[L..R], L=0, R=11:

0 1 2 3 4 5 6 7 8 9 10 11

wartości:

11, 8, 4, 15, 13, 2, 18, 17, 6, 9, 3, 14

start przeglądu:

i→ lewy wskaźnik

prawy wskaźnik ←j

wykrycie pierwszej pary do zamiany:

11, 8, 4, 15, 13, 2, 18, 17, 6, 9, 3, 14

i→

←j

zamiana:

11, 8, 4, 3, 13, 2, 18, 17, 6, 9, 15, 14

kontynuacja przeglądu i wykrycie drugiej pary:

i→

←j

zamiana:

11, 8, 4, 3, 9, 2, 18, 17, 6, 13, 15, 14

dalej, wykrycie trzeciej pary (18,6) i zamiana:

i→

←j

11, 8, 4, 3, 9, 2, 6, 17, 18, 13, 15, 14

kolejna para wykryta po minięciu się wskaźników –

koniec przeglądu

←j i→

zamień A[0] z elementem, na którym zatrzymał się prawy wskaźnik (jego położenie zwraca funkcja partition):

6, 8, 4, 3, 9, 2, 11, 17, 18, 13, 15, 14

[podzadanie lewe]

[podzadanie prawe]

rekurencja

rekurencja

Przypadki szczególne

1. pivot mniejszy od pozostałych elementów:

$A[L] < A[k], k=L+1, \dots, R$:

- indeks j zatrzymuje się na początku tablicy, $j=L$ ($A[L]$ jest wartownikiem), $i=L+1$,
- funkcja partition zwraca L ,
- lewe podzadanie puste ($R=L-1$),
- prawe podzadanie o 1 krótsze niż zadanie oryginalne;

2. pivot większy od pozostałych elementów:

$A[L] > A[k], k=L+1, \dots, R$:

- potrzebny wartownik w $A[R+1] \geq A[k], k=L, \dots, R$,
- dla całego zadania ustawiany jest na początku w $A[n] = \max A[0..n-1]$,
- dla podzadania $A[L..R]$ wartownikiem jest $A[R+1]$, który był pivotem wcześniejszego podziału, zatem $A[R+1] \geq A[k], k=L, \dots, R$ (i dlatego pętle wyszukiują również elementy równe pivotowi i je zamieniają),
- partition zwraca R ,
- lewe podzadanie o 1 krótsze niż całe zadanie,
- prawe podzadanie puste.

Jeśli na wejściu ciąg posortowany rosnąco:

- w wywołaniach rekurencyjnych wszystkie lewe podzadania są puste,
- kolejne prawe o 1 krótsze niż poprzednie.

3. wszystkie elementy jednakowe:

$A[L] = A[L+1] = \dots = A[R]$

Wtedy partition dzieli ciąg na połowy (w przybliżeniu)

Złożoność:

- partition wykonuje $n+1$ porównań (lub n , gdy element w miejscu podziału jest równy pivotowi),
- czas wykonania całego algorytmu zależy istotnie od równowagi podziału na podzadania,
- tej równowagi nie da się zagwarantować w sposób praktycznie zadowalający.

Przypadek najlepszy:

w każdym podziale rozmiary obu podzadań różnią się co najwyżej o 1, np. gdy wszystkie klucze jednakowe:

$$T(n) = 2T(n/2) + n + 1 \Rightarrow T(n) = \Theta(n \log n).$$

Przypadek pesymistyczny:

w każdym podziale jedno z podzadań puste, np. dla ciągu wejściowego różnowartościowego, posortowanego rosnąco:

$$T(n) = \begin{cases} T(n-1) + n + 1, & n > 1 \\ 1, & n \leq 1 \end{cases}$$

Rozwiązanie: $T(n) = 0,5n^2 + 1,5n$. Zatem:

Quicksort ma złożoność kwadratową

(domyślnie chodzi o złożoność pesymistyczną)

Złożoność średnia

Dla złożoności algorytmu nieistotne są konkretne wartości elementów, ale to w jakim są porządku.

Twierdzenie.

Oczekiwana liczba porównań w algorytmie quicksort dla tablicy zawierającej losową permutację liczb $\{1, \dots, n\}$ wynosi

$$A(n) \approx n \ln n \approx 1.39 n \lg n$$

Def. Permutacja danego zbioru jest losowa jeśli jest wynikiem próby losowej, w której każda permutacja tego zbioru jest jednakowo prawdopodobna.

Fakt.

Odchylenie standardowe dla liczby porównań w algorytmie quicksort wynosi $\approx 0,68 * n$.

Zatem jest bardzo małe - liniowego rzędu, podczas gdy wartość oczekiwana jest rzędu $n \lg n$.

Wniosek:

Jeśli na wejściu do algorytmu quicksort dana jest losowa permutacja danych, to prawdopodobieństwo tego, że algorytm będzie wykonywał dużo więcej lub dużo mniej niż $1.4 n \lg n$ porównań jest pomijalne.

Powyższe własności tłumaczą wysoką praktyczną użyteczność quicksortu.

Quicksort w praktyce

1. Małe podzadania (np. $n < 20$) sortuj prostą metodą (np. przez wstawianie). Rekurencja jest nieopłacalna.

Dla quicksort-u eleganckie rozwiązanie:

- wykonaj quicksort, pozostawiając podzadania małe nieposortowane,
- po zakończeniu wywołaj raz insertionsort dla całego ciągu.

Krok drugi jest bardzo szybki, bo ciąg jest już prawie posortowany – w każdym kolejnym nieposortowanym fragmencie wszystkie elementy są większe niż w poprzednim fragmencie.

2. Znieczulanie algorytmu na niesprzyjający rozkład prawdopodobieństwa

(czyli na dużą możliwość wystąpienia ciągów prawie posortowanych):

(A) randomizacja: w każdym podzadaniu, przed przystąpieniem do podziału losuj element i zamień go z $A[L]$. To zapewnia, że

- wszystkie możliwe wielkości podzadań są jednakowo prawdopodobne,
- nikt nie jest w stanie skonstruować ciągu, który jest na pewno przypadkiem pesymistycznym.

Dodatkowy czas na losowania i zamiany jest pomijalny.

(B) "mediana trzech": zamiast losować, wyznacz medianę trzech elementów: $A[L]$, $A[(L+R)/2]$, $A[R]$, i zamień ją z $A[L]$ (ta mediana będzie pivotem)

- optymalnie radzi sobie z ciągiem uporządkowanym,
- nadal można skonstruować dowolnie długie ciągi – przypadki pesymistyczne, które wymagają czasu $\Theta(n^2)$;
- oczekiwana liczba porównań wynosi około $1.2 n \lg n$,
- dodatkowy koszt: 3 porównania dla każdego podzadania, w sumie $\Theta(n)$ - pomijalny.

3. Złożoność pamięciowa quicksortu.

Pamięć robocza jest wykorzystywana niejawnie: jest to stos rekurencji.

Pesymistycznie, np. dla ciągu rosnącego: $\Theta(n)$

Ostrzeżenie:

W podstawowej wersji algorytm quicksort, wywołany dla długiego ciągu prawie posortowanego, może spowodować przekroczenie pamięci przeznaczonej na stos.

Wersja optymalizująca pamięć:

- wykonaj podział aktualnego zadania;
- rekurencyjnie posortuj podzadanie krótsze;
- aktualne zadanie := podzadanie dłuższe;
- jeśli długość aktualnego zadania > 1 to powtarzaj podział, w przeciwnym przypadku zakończ.

Przekształciliśmy quicksort tak:

- *ustawiliśmy kolejność podzadań: najpierw krótsze;*
- *usunęliśmy rekurencję ogonową, czyli podzadanie dłuższe rozwiązujemy iteracyjnie.*

Teraz głębokość stosu wynosi $O(\log n)$ - najgorszym przypadkiem jest gdy rekurencyjnie (czyli tam gdzie narasta stos) wchodzimy do podzadania wielkości połowy zadania poprzedniego.

Fakt.

Istnieje nierekurencyjna wersja bez stosu quicksortu, o takich samych parametrach złożonościowych:

- na stos (intuicyjny) odkładane zawsze prawe podzadanie,
- lewy koniec podzadania na szczycie stosu jest prawym końcem aktualnego zadania, więc nie musi być pamiętany na stosie;
- prawy koniec podzadania pamiętanego na stosie jest "oznaczany" w tablicy za pomocą sprytnej zamiany, która wstawia w to miejsce odpowiednio mały element.

Nie jest ona praktycznie konkurencyjna.

Quicksort, wersja Nico Lomuto

Elementem dzielącym jest ostatni w przedziale, $v=A[R]$

Inv: $A[L..R] = [L \dots i, i+1 \dots j-1, j \dots R-1, R]$
 $\{ \leq v \} \{ > v \} \{ ??? \} v$

LomutoPartition(A, L, R)

$v = A[R];$ // ostatni element jest dzielący

$i = L-1;$

for ($j=L; j<R; j++$)

if ($A[j] \leq v$)

$i = i+1;$

swap ($A[i], A[j]$);

swap ($A[i+1], A[R]$);

return $i+1$;

end.

i	L,j							R
2	8	7	1	3	5	6	4	

po 3 iteracjach:

L,i			j					R
2	8	7	1	3	5	6	4	

iteracja 4: zamiana 1 z 8

L	i			j				R
2	1	7	8	3	5	6	4	

.....

L		i					R,j
2	1	3	8	7	5	6	4

po ostatniej iteracji zamiana 4 z 8:

2	1	3	4	7	5	6	8
---	---	---	---	---	---	---	---

podzadania:

2	1	3		7	5	6	8
---	---	---	--	---	---	---	---

Zalety wersji Lomuto:

- prostszy w implementacji,
- nie wymaga wartownika.

Wady:

- mniej efektywny gdy dużą część ciągu stanowią jednakowe klucze;
- w szczególności ciąg wszystkich kluczy jednakowych stanowi najgorszy przypadek;
- wykonuje znacznie więcej zamian, nawet trzykrotnie.

Pozostałe aspekty analizy złożoności oraz praktyczne uwagi takie same.

Uwaga: metoda Lomuto to rozwiązanie 2-kolorowej wersji znanego zadania o holenderskiej fladze narodowej (3-kolorowej) (Edsger Dijkstra):

- dana jest tablica $A[n]$, $A[i].key \in \{0,1,2\}$ (piksele flagi w 3 kolorach - rozsypały się);
- posortuj A niemalejąco.

Rozwiązanie: jeden przegląd tablicy. Szczegóły? - miłe ćwiczenie. Poprawność? Zastosuj niezmienniki.

Quicksort - podział na 3 części

jako wersja zadania o fladze: podziel tablicę na:

- elementy mniejsze niż pivot,
- równe pivotowi,
- większe niż pivot.

Dalsza rekurencja osobno w części pierwszej i w trzeciej.

Efektywny gdy wiele kluczy się powtarza.